

SECURE AWS CONTENT DELIVERY PLATFORM

Secure content access using authentication, authorization and temporary controlled access mechanisms

Amazon Cognito

API Gateway

AWS Lambda

Private S3

Pre-Signed URLs

Prepared by

Adhithyan Sivaraman T

GitHub: github.com/Adhithyan-10

LinkedIn: [linkedin.com/in/adhithyan-sivaraman-t-399b5b362](https://www.linkedin.com/in/adhithyan-sivaraman-t-399b5b362)

TABLE OF CONTENTS

01

Project Overview

Problem statement & solution architecture

02

Real World Scenario

Netflix-style secure content delivery use case

03

Architecture Overview

End-to-end system design walkthrough

04

AWS Services Used

Services, purpose, and design rationale

05

Authentication Flow

Cognito Hosted UI & OAuth Authorization Code

06

Token Exchange & JWT

Backend token exchange and JWT validation

07

Secure API Protection

API Gateway + JWT Authorizer in action

08

Secure S3 Access

Private bucket, access denial & pre-signed URLs

09

IAM Security

Least privilege IAM policy configuration

10

Challenges & Solutions

CORS, JWT issues & configuration fixes

11

Security Principles

Key lessons in cloud security design

12

Future Enhancements

WAF, Secrets Manager & CloudWatch monitoring

13

Interview Q&A

10 key questions with detailed answers

14

Conclusion

Summary & portfolio statement

Project Overview



BEFORE — The Problem

Public API Exposure

APIs were publicly accessible — no authentication or authorization gates in place.

No Access Control

Any user could call backend endpoints directly, bypassing security checks.

Direct S3 Access Risk

S3 objects were potentially reachable with a direct URL — no bucket-level protection.

No Identity Verification

No mechanism to verify who the user is before serving sensitive content.



AFTER — The Solution

Amazon Cognito

User Pool with Hosted UI handles authentication, user management and OAuth 2.0 flows.

API Gateway + JWT Authorizer

Every API request is validated against a Cognito-issued JWT token before reaching Lambda.

Private S3 + IAM Roles

Bucket has Block Public Access enabled; Lambda accesses it only via a scoped IAM role.

Pre-Signed URLs

Lambda generates time-limited, cryptographically signed S3 URLs — no direct bucket exposure.

02 • REAL WORLD SCENARIO

Real World Scenario



Netflix-Style Secure Video Delivery

Consider a streaming platform where video content (stored in S3) must only be accessible to paid, authenticated users. A direct S3 URL would expose all content to anyone with the link — including bots, scrapers, and unauthorized users.

WITHOUT SECURITY



Direct S3 URL works for anyone — no login required



Scraper bots can download and redistribute content freely



No audit trail of who accessed what and when



URLs never expire — shared links work indefinitely



No way to revoke access for a suspended user

WITH THIS ARCHITECTURE



User must authenticate via Cognito — no token, no access



JWT is validated on every single API request by API Gateway



Lambda generates a time-limited URL (e.g. 15 minutes) per request



Direct S3 access is blocked — content only reachable via signed URL



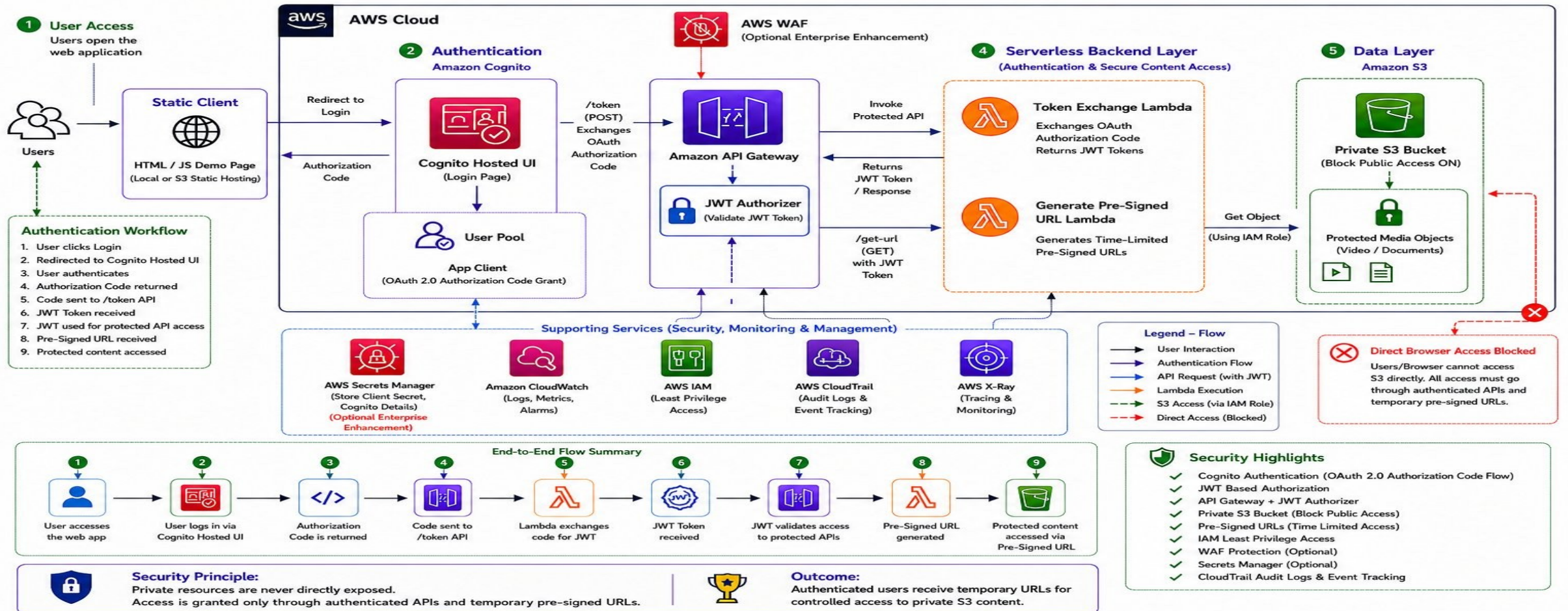
CloudTrail logs every API call and S3 access for full auditability

03 • ARCHITECTURE OVERVIEW

Architecture Overview

Secure AWS Content Delivery Platform – Architecture Diagram

Authentication • Authorization • Secure APIs • Private S3 • Pre-Signed URLs • Temporary Secure Access



03 • ARCHITECTURE WALKTHROUGH

End-to-End Request Flow

User Access

1

User opens the web app and clicks Login.

Cognito Hosted UI

2

Redirected to Cognito's managed login page — email/password authentication.

Authorization Code

3

After login, Cognito redirects back with a one-time authorization code.

Token Exchange

4

Frontend sends code to /token API. Lambda exchanges it with Cognito for JWT tokens.

JWT Validation

5

For every protected API call, the JWT Authorizer in API Gateway validates the token.

Lambda Invocation

6

Validated request reaches Lambda, which uses its IAM role to call S3.

Pre-Signed URL

7

Lambda generates a time-limited signed URL for the S3 object.

Content Access

8

User's browser uses the pre-signed URL to securely stream/download content.

AWS Services Used

Service	Purpose	Why Used
Amazon Cognito	User identity and authentication management	Provides managed OAuth 2.0 Hosted UI, User Pools, App Clients and JWT token issuance without building custom auth.
API Gateway	HTTP API frontend — routes and protects all requests	Integrates natively with Cognito JWT Authorizer to block unauthenticated requests before they reach Lambda.
AWS Lambda	Serverless backend logic	Handles token exchange with Cognito and generates pre-signed S3 URLs on authenticated request — no servers to manage.
Amazon S3	Private object storage for protected content	Block Public Access ensures content is never directly reachable; only accessible via Lambda-generated signed URLs.
AWS IAM	Role-based access control for Lambda	Lambda's execution role is granted only s3:GetObject — enforcing least privilege so it cannot modify or delete objects.
JWT Authorizer	Validates JWT on every incoming API request	Decodes and verifies the Cognito-issued id_token against the JWKS endpoint — rejects expired or tampered tokens.
CloudTrail	Audit logging for all AWS API calls	Every Lambda invocation, S3 access, and Cognito event is logged — providing a full tamper-evident audit trail.

Cognito Configuration

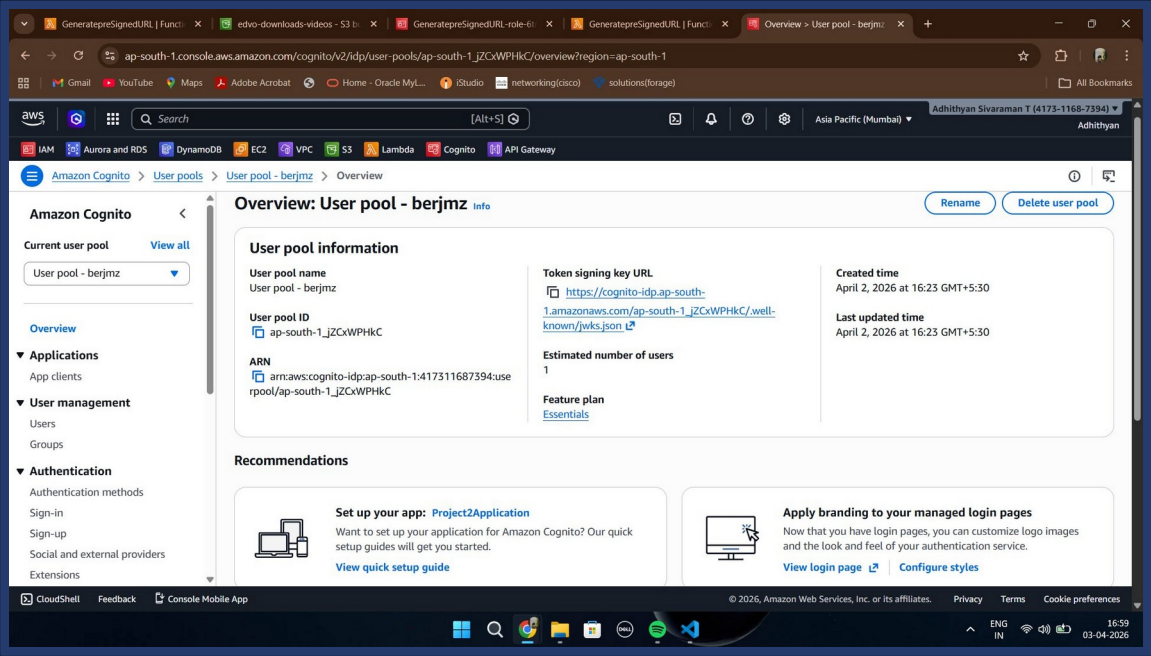


Figure 1 — Cognito User Pool Overview

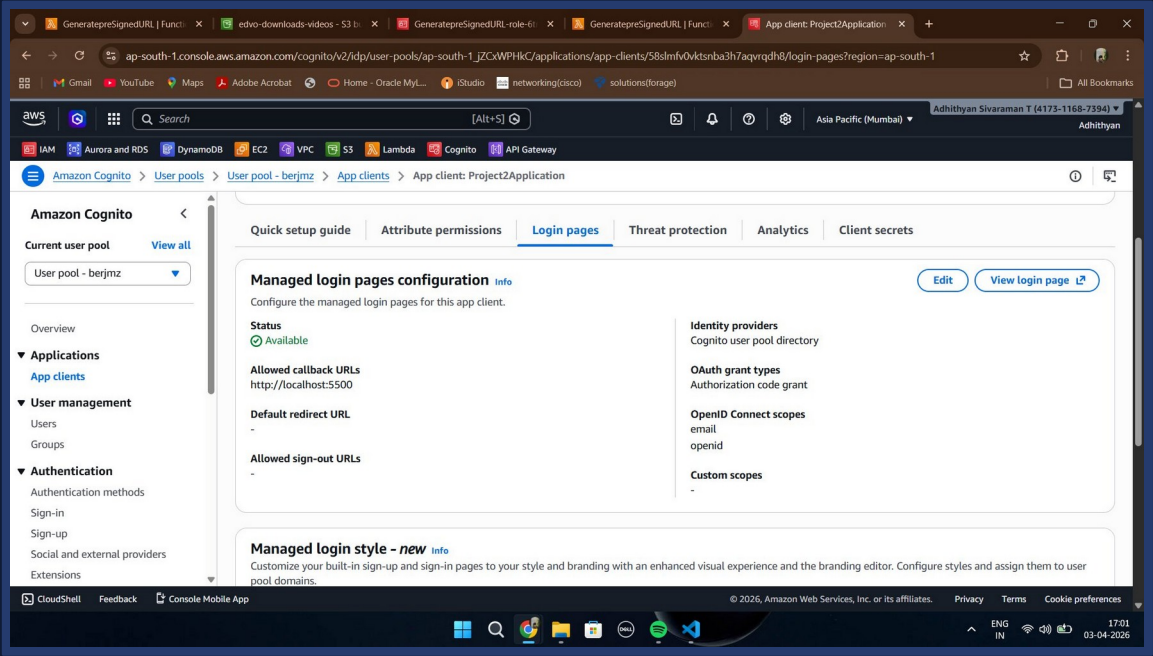


Figure 2 — App Client with OAuth 2.0 Authorization Code Grant enabled

- User Pool:

An Amazon Cognito User Pool was created in ap-south-1. It stores user credentials and manages sign-in flows.
- App Client:

The App Client is configured with Authorization Code Grant, allowing OAuth 2.0 flows with email/openid scopes.
- Callback URL:

Development callback URL configured for local testing environment. The token exchange happens server-side — client secret stays hidden.

05 • AUTHENTICATION FLOW

Hosted UI & Authorization Code

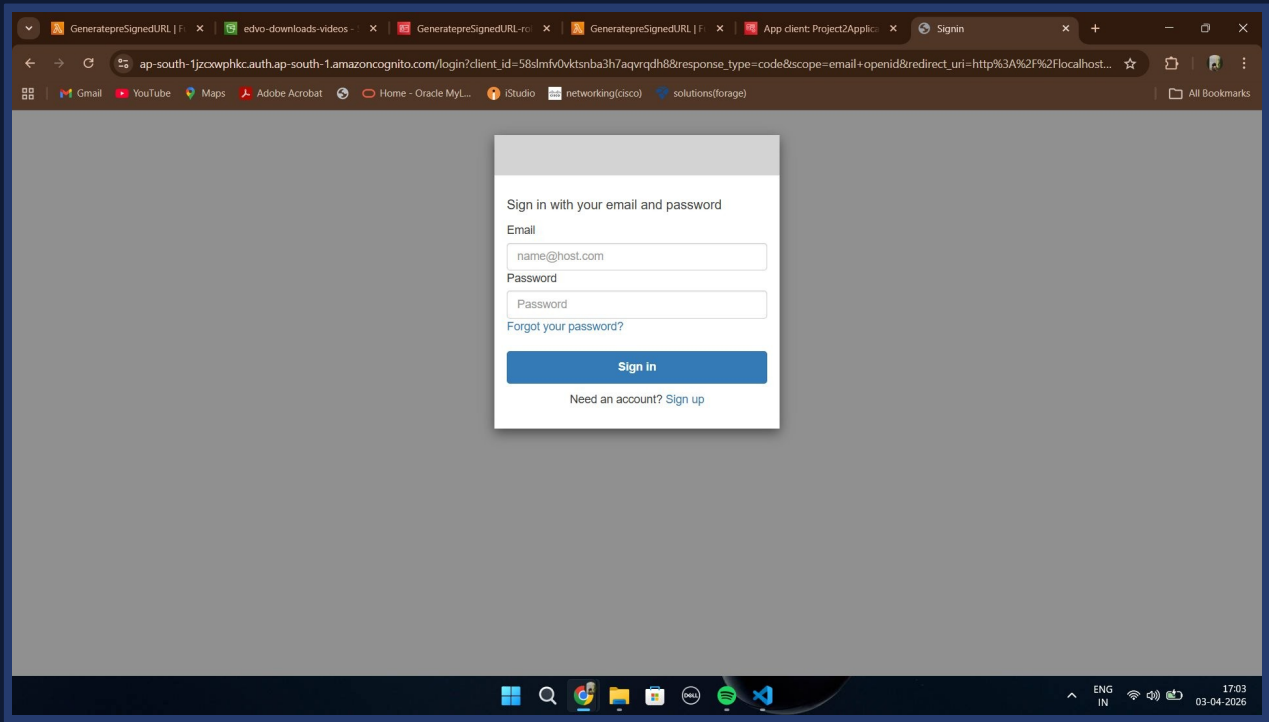


Figure 3 — Cognito Hosted UI Login Page

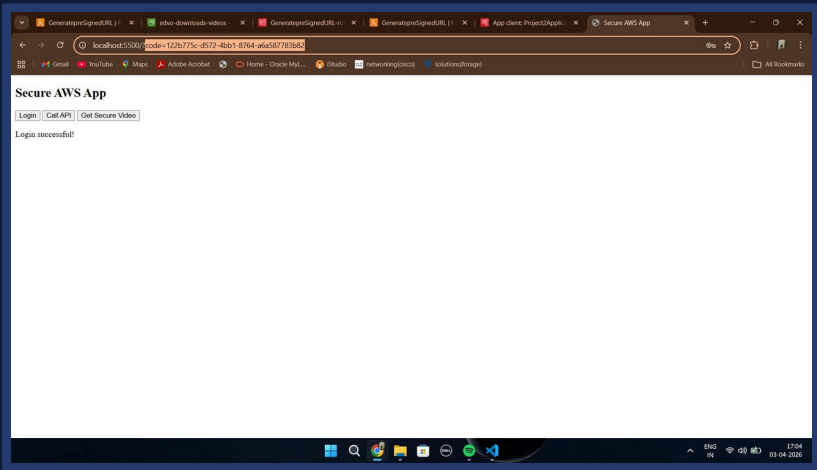
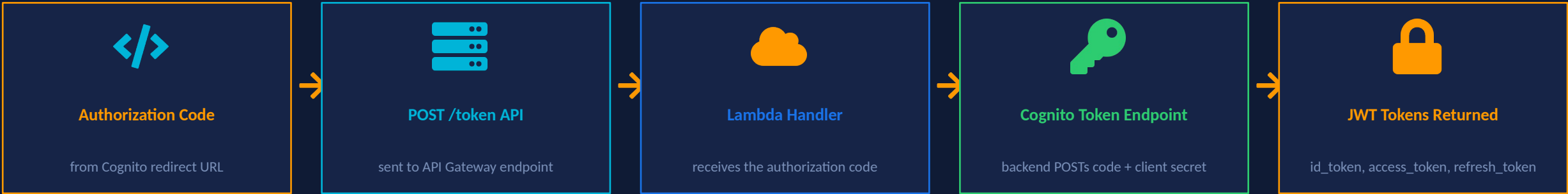


Figure 4 — Authorization Code appended to redirect URL after login

Hosted UI:	Cognito manages the login page — no custom auth UI needs to be built. The URL includes the client_id, response_type=code, scope, and redirect_uri.
Authorization Code:	After successful login, Cognito redirects the user back to the callback URL with a short-lived authorization code appended as a query parameter (?code=...).
Why Code, Not Token?:	The code grant pattern prevents tokens from appearing in browser history or server logs. The actual token exchange happens server-side via Lambda.

Token Exchange & JWT Flow



Why Backend Token Exchange?

The client secret attached to the Cognito App Client must NEVER be exposed in frontend JavaScript. If the token exchange happened in the browser, the secret would be visible in the source code. By routing it through Lambda, the secret stays in the backend environment — inaccessible to users.

What's Inside the JWT?

sub Unique user identifier from Cognito	email User's verified email address	exp Token expiration timestamp (Unix)	iss Issuer — Cognito User Pool URL	token_use id or access — defines token purpose
---	---	---	--	--

Secure API Protection

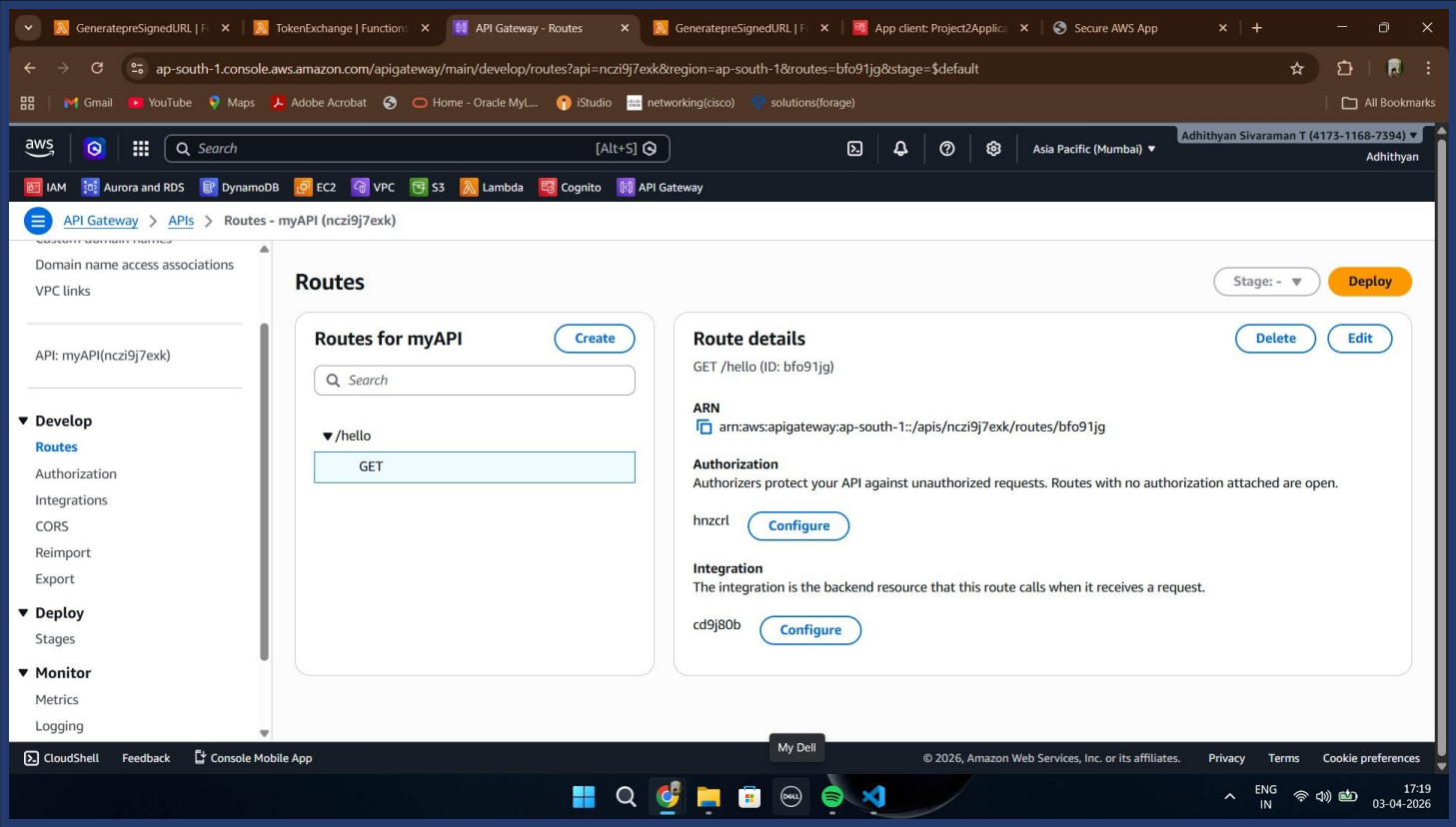


Figure 5 — API Gateway route /hello with JWT Authorizer attached

Route: GET /hello

The protected endpoint is configured in API Gateway. Without a JWT Authorizer attached, it is open to the world.

JWT Authorizer

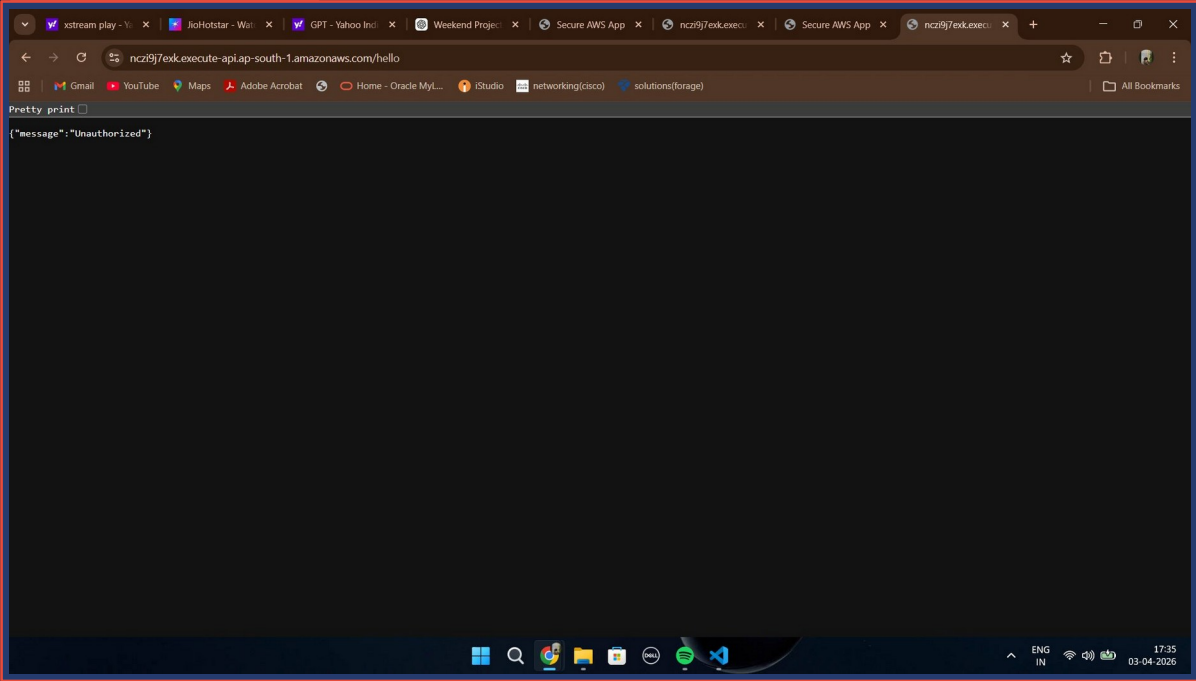
The authorizer is configured with the Cognito User Pool's JWKS endpoint. It validates the token signature, issuer, and expiry on every request.

Integration (Lambda)

Only after successful JWT validation does API Gateway forward the request to the backend Lambda function.

Unauthorized vs Authorized API Access

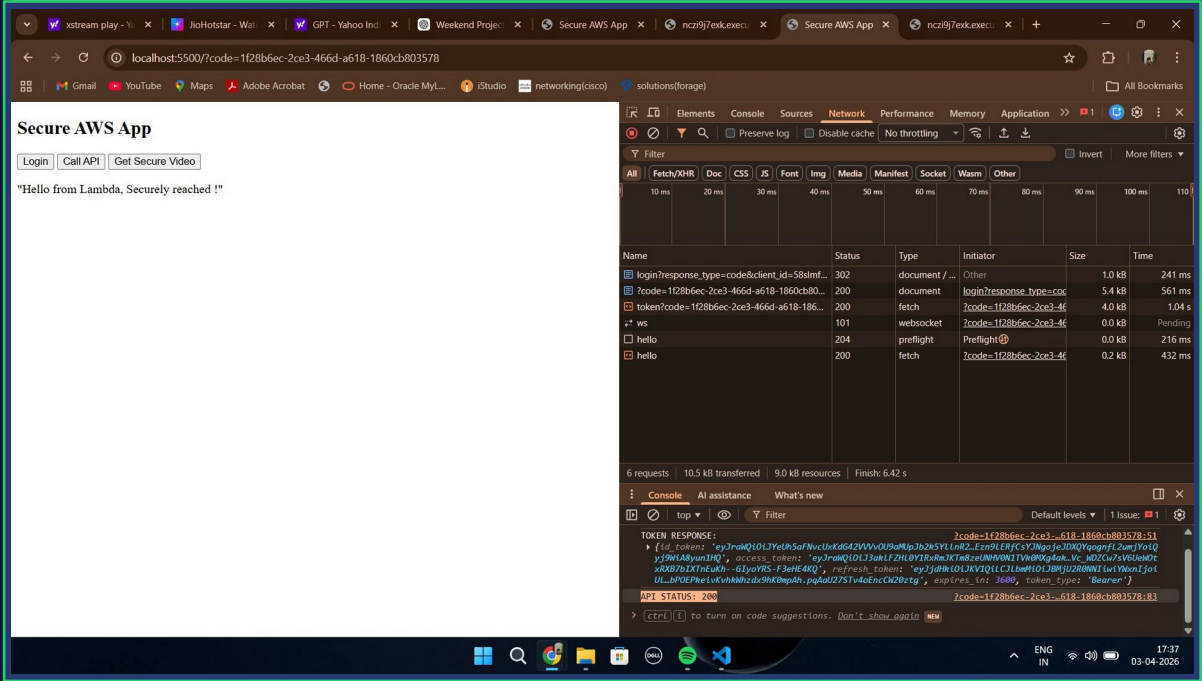
401 UNAUTHORIZED



The screenshot shows a web browser with the address bar displaying `ncz977exk.execute-api.ap-south-1.amazonaws.com/hello`. The page content shows a JSON response: `{"message": "Unauthorized"}`. The browser's developer tools are open, showing the network tab with a single request that resulted in a 401 status.

Direct browser access to API endpoint — no JWT token provided

200 SUCCESS



The screenshot shows a web browser with the address bar displaying `localhost:5500/?code=1f28b6ec-2ce3-466d-a618-1860cb803578`. The page content shows "Hello from Lambda, Securely reached !". The browser's developer tools are open, showing the network tab with a single request that resulted in a 200 status. The console shows a message: "Hello from Lambda, Securely reached !".

Lambda responds with 200 after JWT is validated by API Gateway Authorizer



Security in Action:

The JWT Authorizer acts as a gate before the Lambda function is ever reached. Expired, missing, or tampered tokens all result in a 401 response — Lambda is never invoked, reducing attack surface and compute costs simultaneously.

Private S3 Bucket Configuration

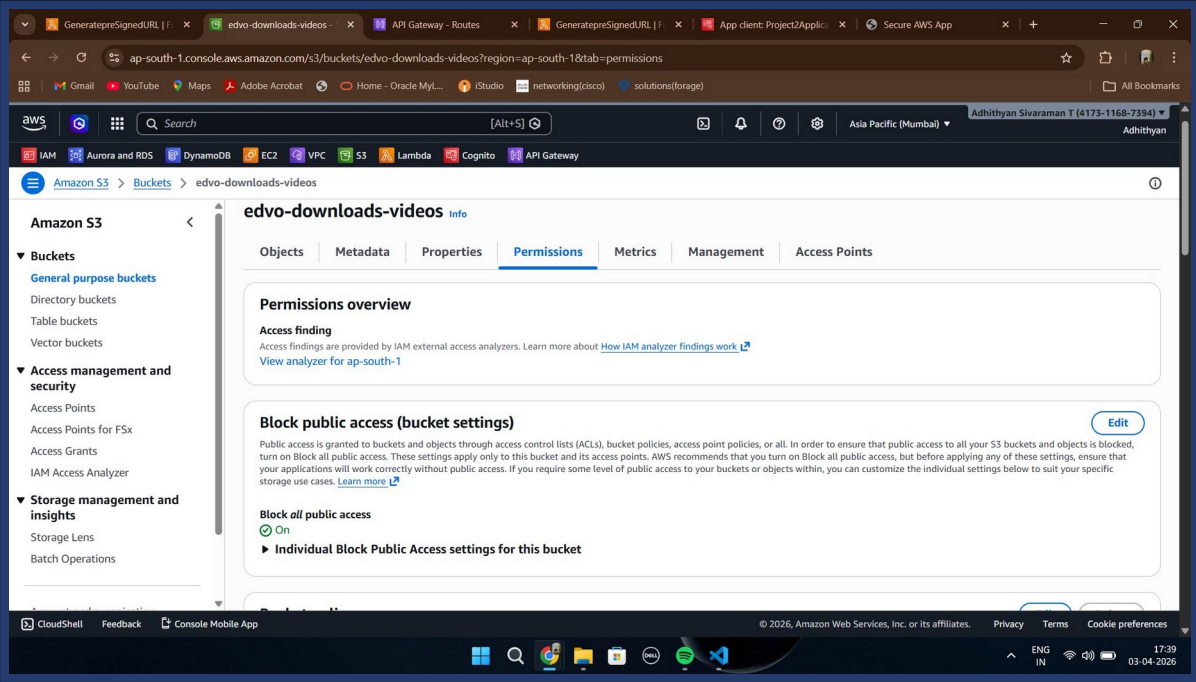


Figure 6 — S3 Bucket (edvo-downloads-videos) with Block All Public Access: ON

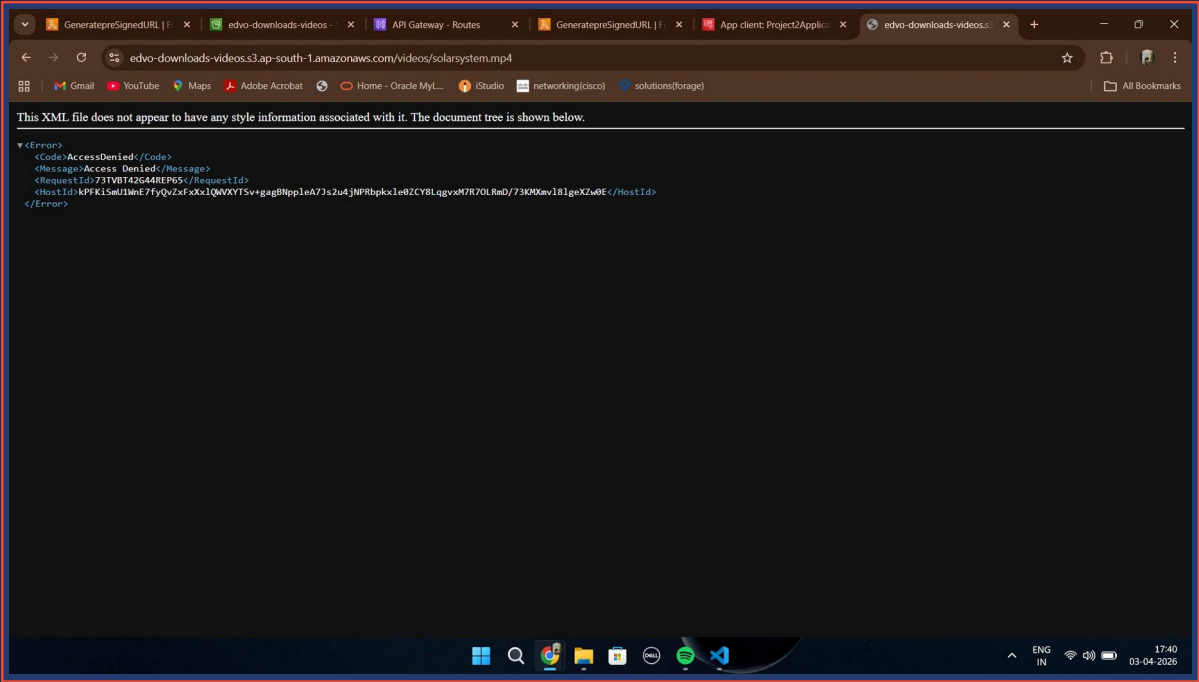


Figure 7 — Direct S3 URL returns AccessDenied — security validation successful

- Block All Public Access: ON:

This single setting prevents any S3 object from being reachable via a public URL, regardless of object-level ACLs.
- Direct URL: AccessDenied:

Attempting to access the S3 object directly returns an XML AccessDenied error — confirming the bucket is truly private.
- Forced Pre-Signed Flow:

The only valid access path is through Lambda, which holds the IAM role with s3:GetObject permission for this specific bucket.

08 • PRE-SIGNED URL FLOW

Pre-Signed URL Generation & Access

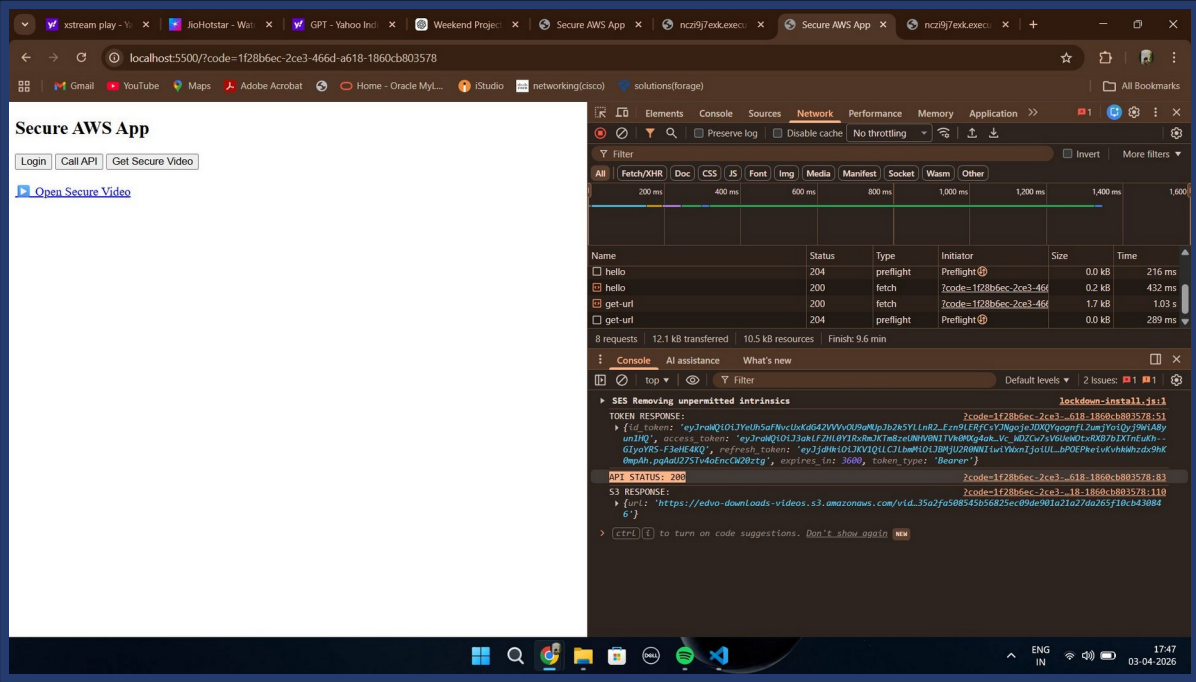


Figure 8 — Pre-signed S3 URL returned in API response (with S3 RESPONSE visible in console)

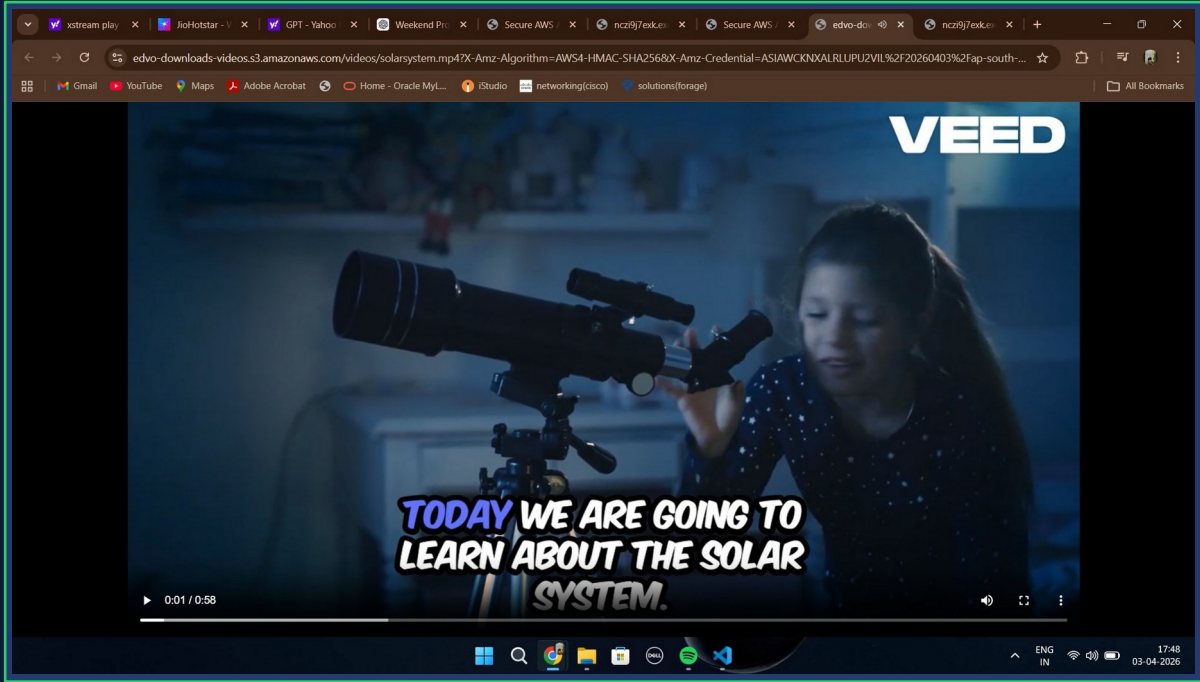


Figure 9 — Private video (solarsystem.mp4) accessible via pre-signed URL — time-limited access

How Pre-Signed URLs Work:	Lambda calls boto3's generate_presigned_url() with the bucket name, object key, and expiry duration. AWS embeds the credentials cryptographically in the URL itself.	4
Time-Limited Access:	The URL includes X-Amz-Expires in the query string. After expiry (e.g. 900 seconds), S3 rejects the request — even with the exact same URL.	
No Bucket Policy Change:	The bucket remains fully private. Pre-signed URLs work because Lambda's IAM role has s3:GetObject — the URL borrows that permission temporarily.	

09 • IAM SECURITY

IAM — Least Privilege Access

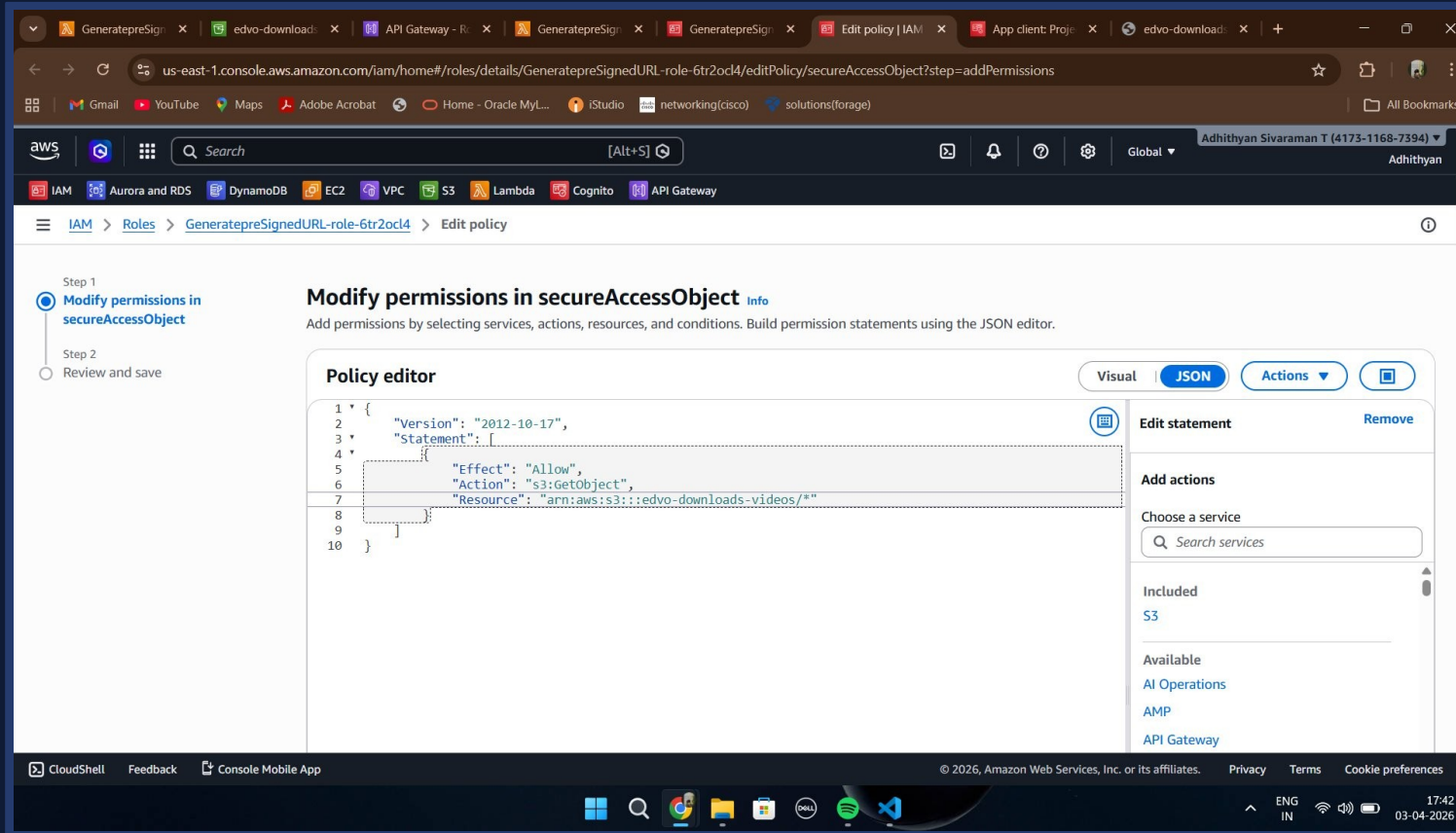


Figure 10 — IAM Policy granting only `s3:GetObject` on the `edvo-downloads-videos` bucket

Exact Action

Effect: Allow, Action: `s3:GetObject`. No `s3:PutObject`, `s3:DeleteObject`, or `s3:ListBucket` — Lambda can read only.

Specific Resource

Resource: `arn:aws:s3:::edvo-downloads-videos/*` — scoped to this bucket only. Not `*`.

Role, Not Keys

Lambda assumes an IAM Role at runtime. No access keys stored in code or environment variables.

Audit Trail

Every IAM role assumption and `S3:GetObject` call is logged in CloudTrail automatically.

10 • CHALLENGES FACED

Challenges & How I Fixed Them

01 Unauthorized Access Despite Valid Token



Problem:

API kept returning 401 even after the token exchange succeeded.



Fix:

Resolved by configuring the correct Cognito issuer URL and validating JWT settings.

02 CORS Errors on Token Exchange



Problem:

Browser blocked the POST /token request due to missing CORS headers.



Fix:

Added explicit CORS configuration in API Gateway for the /token route — allowed origins, methods (POST), and headers (Content-Type, Authorization).

03 Wrong Callback URL Configuration



Problem:

Cognito rejected the redirect after login with an OAuth error.



Fix:

The callback URL in the Cognito App Client must exactly match the redirect_uri in the frontend request — including protocol, host, and port. Development callback URL configured for local testing environment.

04 Pre-Signed URL Expired Too Quickly



Problem:

URL expired before the video could fully load in the browser.



Fix:

Increased the ExpiresIn parameter in generate_presigned_url() from 60 to 900 seconds. For production, the right value depends on expected file size and connection speed.

Security Principles Learned



Authentication vs Authorization

Authentication asks 'who are you?' (Cognito answers this). Authorization asks 'what are you allowed to do?' (JWT Authorizer + IAM answers this). Both are required.



Never Expose Private Resources

Private S3 objects must never be directly linked. All access must route through an authenticated API that validates identity before granting temporary access.



Temporary Access Pattern

Pre-signed URLs embody the principle of time-limited access — credentials that expire by design reduce the blast radius of a leaked URL significantly.



Least Privilege by Default

IAM roles should grant only the exact actions needed. A Lambda that reads S3 objects does not need s3:PutObject, s3:DeleteObject, or s3:ListBucket.



Backend Token Exchange

Client secrets must never leave the server. The OAuth Authorization Code flow with backend exchange ensures the Cognito App Client secret stays hidden from browser clients.



Validate on Every Request

JWTs are stateless — there is no session. The JWT Authorizer re-validates the signature and expiry on every single API request, providing continuous verification.

12 • FUTURE ENHANCEMENTS

Future Enhancements



AWS WAF

Web Application Firewall

Attach WAF to API Gateway to block SQL injection, XSS, and rate-limit by IP. Essential for production APIs exposed to the public internet.



AWS Secrets Manager

Secure credential rotation

Move the Cognito client secret out of Lambda environment variables and into Secrets Manager with automatic rotation on a schedule.



CloudWatch Monitoring

Observability & alerting

Add Lambda error rate alarms, API Gateway 4xx/5xx dashboards, and S3 access anomaly detection to catch security events in real time.



CloudFront Distribution

Global CDN + signed cookies

Place CloudFront in front of S3 with signed cookies or signed URLs issued by CloudFront — enabling geo-restriction and edge caching of content.



Auto URL Regeneration

Frontend refresh logic

Implement a frontend timer that automatically requests a new pre-signed URL before expiry — providing seamless access without re-authentication.



User Groups & Roles

Fine-grained authorization

Use Cognito User Groups to define roles (admin, premium, free). Lambda reads group claims from the JWT and returns different content tiers accordingly.

Interview Questions & Answers



What is the difference between Authentication and Authorization?



Authentication verifies identity — Cognito does this via the Hosted UI login. Authorization decides what the authenticated user can access — the JWT Authorizer and IAM roles handle this. Both are required for secure systems.



Why use Authorization Code flow instead of Implicit flow?



The Implicit flow returns tokens directly in the URL, exposing them to browser history and server logs. Authorization Code flow keeps tokens out of the URL and exchanges the code server-side, keeping the client secret hidden.



What happens if a JWT token is expired when hitting the API?



The JWT Authorizer in API Gateway validates the exp claim. An expired token results in a 401 Unauthorized response immediately — Lambda is never invoked. The user must re-authenticate to get a fresh token.



Why must token exchange happen in Lambda and not the browser?



The Cognito App Client secret is required for the token exchange request. Placing this logic in the frontend would expose the secret in JavaScript source code. Lambda keeps it server-side and secure.



How does a pre-signed URL grant access without making S3 public?



Pre-signed URLs embed the IAM credentials of the signer (Lambda's role) cryptographically in the URL. S3 validates these credentials against the policy at request time — granting temporary access without changing the bucket's public access settings.

Interview Questions & Answers



What is the Principle of Least Privilege and how did you implement it?



Least privilege means granting only the permissions required to perform a specific task. Lambda's IAM role was granted only `s3:GetObject` on the specific bucket ARN — not `s3:*` or even `s3:ListBucket`.



What are JWKS and how does the JWT Authorizer validate tokens?



JWKS (JSON Web Key Set) is a public endpoint published by Cognito containing the RSA public keys used to sign JWTs. The JWT Authorizer fetches the key matching the `kid` header in the token and verifies the signature — no database lookup needed.



How would you prevent a leaked pre-signed URL from being misused?



Set a short expiry (15-30 minutes). In production, bind the URL to the requester's IP using S3 condition keys (`aws:SourceIp`). Use CloudFront Signed URLs with geographic restrictions for stronger control.



What is the difference between `id_token` and `access_token` in Cognito?



The `id_token` contains user identity claims (email, sub, phone). The `access_token` is used to call Cognito APIs (e.g. get user info). For API Gateway JWT Authorizer, the `id_token` is used to validate the user's identity.



How would you scale this architecture for 10,000 concurrent users?



Lambda auto-scales by default. API Gateway handles horizontal scaling automatically. Increase Cognito's token refresh intervals to reduce auth traffic. Add CloudFront in front of API Gateway for edge caching. Use SQS to buffer Lambda invocations under burst load.

Conclusion

Designed and implemented a fully serverless, secure AWS content delivery architecture that demonstrates real-world application of identity management, API security, and private resource access control. The system enforces authentication through Amazon Cognito, validates every request via a JWT Authorizer in API Gateway, and delivers private S3 content exclusively through time-limited pre-signed URLs — ensuring that no resource is ever directly exposed.

7+

AWS Services Integrated

0

Private S3 Access Enforced

100%

APIs JWT Protected

Full

Serverless Architecture

What This Project Demonstrates



Identity & Access Management — Cognito, OAuth 2.0, JWT



API Security Design — JWT Authorizers, CORS, route protection



Cloud Storage Security — Private S3, IAM roles, pre-signed URLs



Serverless Architecture — Lambda, API Gateway, event-driven design